

NES555 — Lab 1: Packet Crafting & Network Analysis

Lab title: Lab 1 — Packet Crafting with Scapy & Network Analysis

Course: NES555 — Ethical Hacking Lab

Version: 2025-10-08

Target VMs (from course lab setup):

- **Kali Linux** (attacker) — configured per the Lab Setup Guide.
- **BWA / OWASP Broken Web App (BWA)** — target web application VM.
- **Metasploitable** — vulnerable target VM.

Important: This lab uses only the VM images and network topology described in the NES555 — Lab Setup Guide. Make sure you completed the setup steps (Host-only adapter `vboxnet0` or `nes555_net`, snapshots, and IP addressing) before starting.

Learning objectives

By the end of this lab you will be able to:

1. Build and inspect custom network packets using **Scapy** (layer-by-layer).
 2. Send crafted packets and interpret responses (ICMP and TCP SYN behaviors).
 3. Capture network traffic with **tcpdump** and analyze it using **Wireshark**.
 4. Use simple Scapy-based scans and verify open services on target VMs.
 5. Practice safe, isolated testing on lab VMs only.
-

Estimated time

60–90 minutes

Prerequisites (host & VM state)

- Completed the **Lab Setup Guide** (Kali, BWA, Metasploitable created and attached to the Host-only network).

- All three VMs are powered on and reachable on the host-only network.
 - Know each VM's host-only IP. Typical example addresses used in the exercises below (replace with the actual addresses from your lab):
 - **Kali:** 192.168.68.254
 - **BWA (OWASP):** 192.168.68.12
 - **Metasploitable:** 192.168.68.100
 - You are logged in to **Kali** as `root` (or a user with `sudo` privileges).
 - Scapy, `tcpdump`, and Wireshark are installed on Kali (default Kali images include these; otherwise `apt update && apt install -y scapy tcpdump wireshark`).
-

Lab safety & rules (repeat before you start)

- Target only the VMs provided for this course (BWA, Metasploitable) — do not scan or attack any hosts outside the host-only lab network.
 - Do not enable bridged networking during exercises unless the instructor explicitly authorizes it.
 - Restore VMs to the `baseline` snapshot if you break the environment.
-

Background — Why packet crafting and network analysis?

Packet crafting and network analysis are foundational skills for any ethical hacker or penetration tester. Crafting packets lets you emulate attackers, testers, or faulty devices by manipulating packet headers and payloads at each protocol layer (Ethernet, IP, TCP/UDP, ICMP, etc.). This teaches how devices and network appliances interpret traffic, how fingerprinting works, and how attackers can bypass or confuse defenses by altering normal protocol fields.

Scapy is a powerful Python-based packet manipulation library that allows you to build, send, sniff, and dissect network packets programmatically. It is widely used in research, testing, and CTFs because of its combination of power and flexibility.

`Tcpdump` (and its GUI counterpart Wireshark) allow you to capture and inspect real network traffic. `Tcpdump` is excellent for quick captures and scripting; Wireshark provides deep, human-friendly packet dissection with filtering and follow-stream features.

Ethical notes:

- Always operate within the scope of an authorized lab. Packet crafting can be disruptive. Use rate limiting and avoid unnecessary noise on shared networks.
 - Document your actions and keep captures for reproducibility and grading.
-

Deliverables (what to submit)

- A short lab report (one page) containing:
 1. Your host-only IP addresses for Kali, BWA, and Metasploitable.
 2. A screenshot (or pcap) showing an ICMP exchange from Kali → BWA created with Scapy.
 3. A brief explanation of a SYN scan result showing port 80 open on BWA (include the Scapy command used and observed response).
 4. One interesting Wireshark screenshot showing either HTTP or SMB traffic captured during the lab and one sentence describing what you observed.
-

Tasks (step-by-step)

Preparation: open terminals and verify IPs

1. Start **Kali**, **BWA**, and **Metasploitable** VMs (if not already running).
2. On Kali, open a terminal and confirm host-only IP: `ip a` or `ip addr show` and note the IP of the host-only interface (e.g., `eth0` or `enp0s3`).
3. Ping the targets to confirm reachability:
 - `ping -c 3 192.168.68.12` (BWA)
 - `ping -c 3 192.168.68.100` (Metasploitable)

If pings fail, check VirtualBox Host-only adapter settings and ensure both VMs are attached to the same host-only network.

Part A — Creating packets with Scapy (build & inspect)

1. Start Scapy on Kali:
 - Open a terminal and run: `scapy` (or `sudo scapy`)
2. At the Scapy prompt, list available layers and commands:
 - `ls()` — lists protocol classes available
 - `lsc()` — lists available Scapy commands
3. Build a simple IP packet object and inspect it:
 - `ip = IP(ttl=10)`
 - `ip` (shows packet summary)
 - `ip.dst` — shows destination (initially may be unset)
4. Set destination and source addresses, then check:

- `ip.dst = "192.168.68.12" # set BWA`
- `ip.src = "192.168.68.254" # set Kali (example)`
- `ip` (verify src/dst/ttl)
- 5. Remove TTL to let Scapy populate default per RFC:
 - `del(ip.ttl)`
 - `ip.ttl` (verify it shows default value; Scapy may re-calculate on send)
- 6. Add a TCP layer on top of IP and create a TCP object with ports:
 - `tcp = TCP(sport=1025, dport=80)`
 - `pkt = ip/tcp` (creates a stacked packet)
 - `pkt.show()` (displays full layer details)
- 7. Optionally add an Ethernet layer (if analyzing at L2):
 - `eth = Ether()`
 - `pkt2 = eth/ip/tcp`
 - `pkt2.show()`

Checkpoint: save a screenshot of `pkt.show()` for your report.

Part B — Sending crafted packets and reading responses

1. From the Scapy prompt, send a crafted ICMP echo request and wait for a response:
 - `packet = sr1(IP(dst="192.168.68.12")/ICMP()/ "HELLO_SCAPY")`
 - `packet` (inspect returned response)
 - Note: `sr1()` sends one packet and returns the first answer; it requires root privileges.
2. Examine the returned packet contents:
 - `packet.show()`
3. Perform a simple single-port SYN probe (TCP) to test port 80 on BWA:
 - `synr = sr1(IP(dst="192.168.68.12")/TCP(dport=80, flags="S"))`
 - Inspect `synr`:
 - If you receive a TCP packet with flags `SA` (SYN+ACK), the port is open.
 - If you receive `R` (RST) or no reply, the port may be closed or filtered.
4. Record the observed behavior (open/closed/filtered) for port 80 and include it in your lab report.

Notes:

- Use `sr()` to send multiple packets and collect responses when needed.
 - Be mindful of rate: do not flood the network.
-

Part C — Capturing traffic with tcpdump (on Kali)

1. Open a new terminal window on Kali.
2. Start a capture on the host-only interface (replace `eth0` with the actual interface name):

- `sudo tcpdump -i eth0 -s 0 -w lab1_capture.pcap`
 - Leave the capture running while you perform the Scapy send steps above.
 - 3. In another terminal, repeat the Scapy ICMP and SYN tests from Part B so tcpdump will record them.
 - 4. Once done, stop tcpdump with `Ctrl+C`. You should have `lab1_capture.pcap` in your current directory.
-

Part D — Analyzing with Wireshark

1. Launch Wireshark on Kali: `sudo wireshark` (or open non-root if configured).
 2. Open the file `lab1_capture.pcap` (File → Open).
 3. Use display filters to focus your analysis:
 - `icmp` — show only ICMP packets
 - `tcp.port == 80` — show TCP traffic related to port 80
 - `http` — filter for HTTP traffic (if BWA served HTTP during the lab)
 4. Analyze a captured ICMP echo request and reply. Use the packet details pane to inspect headers (IP, ICMP, payload).
 5. For your SYN probe, locate the SYN packet and corresponding SYN/ACK (or RST). Right-click a TCP packet and select **Follow** → **TCP Stream** to view the conversation.
 6. Take one screenshot showing either the ICMP exchange or evidence of a SYN/SYN-ACK handshake to include in your report.
-

Extra exercises (optional)

- Use Scapy to craft a TCP packet with an unusual flag combination (e.g., FIN+PSH) and observe the target response.
 - Use `tshark -r lab1_capture.pcap -Y "http"` to extract HTTP objects.
 - Try `nmap -sS -p80 192.168.68.12` from Kali and compare results with the Scapy SYN probe.
-

Questions

Part A — Creating packets with Scapy (build & inspect)

1. Explain, in your own words, what happens to the IP TTL field when you `del(ip.ttl)` in Scapy and then send the packet. Why does Scapy behave this way?
2. Show (or describe) two differences between building a packet with `IP()/TCP()` versus `Ether()/IP()/TCP()` in Scapy. When is the Ethernet layer necessary?

3. Suppose you set `ip.src` to an IP that is not assigned to any interface in the lab. What will the target likely see as the source IP? Explain.

Part B — Sending crafted packets and reading responses

1. You sent a SYN packet using Scapy and received a packet with flags `SA` (SYN+ACK). What does this imply about the target port state, and what ethical considerations should you mention when reporting this finding?
2. If your ICMP echo request with a custom payload